

GROUP 01 · CAPSTONE PROJECT

Health Insurance AI Copilot

Claims Eligibility · Policy Q&A · Prior Authorization · Member Cost Estimation

Domain	Health Insurance
Format	End-to-End AI Copilot
Stack Hint	LangChain · FastAPI · FAISS · Streamlit · Docker · AWS
Duration	Day 19 & 20 — Full Capstone
Team Size	4–5 Participants
Type	Problem Statement Architecture Reference Evaluation Rubric

Enterprise AI Training Program · 15-Day Bootcamp Series

1. Project Overview

You are tasked with building a production-grade **Health Insurance AI Copilot** — an intelligent conversational assistant that serves insurance members, claims reviewers, and customer support agents. The system must be able to answer natural-language questions about health insurance policies, coverage terms, claims eligibility, prior authorization, drug formularies, and member cost-sharing.

This is a full end-to-end engineering capstone. Your team is responsible for all layers: data ingestion, vector search, LLM agent design, API development, UI, safety guardrails, evaluation, containerization, and live cloud deployment.

Domain	Health Insurance
User Personas	Policyholders · Claims Reviewers · Customer Support Agents
Core Challenge	RAG over heterogeneous insurance documents + multi-tool agent reasoning
Final Deliverable	Live URL (AWS) + Streamlit UI + FastAPI + FAISS + Evaluation Scorecard
Duration	Day 19 & 20 (Full capstone + deployment demo)
Team Size	4–5 participants

2. Problem Statement

Health insurers handle thousands of member queries every day across channels — phone, web portal, mobile app, and agent desks. Despite this volume, most queries are repetitive and document-bound: members want to know whether a procedure is covered, how much they will owe, whether they need prior authorization, or which drugs are on their formulary.

The problem is that policy documents are lengthy (50–100+ pages), written in legal-dense language, and vary significantly across plan tiers. Support agents spend excessive time manually cross-referencing documents, leading to slow response times, errors, and poor member satisfaction. Members attempting self-service give up because the information is buried and hard to find.

■ Your Mission

Build an AI-powered copilot that allows a member or agent to ask a natural-language question and receive an accurate, cited, and safe answer — grounded in the actual policy documents — within 4 seconds, without hallucinating coverage that does not exist.

The system must identify the relevant plan tier, retrieve the correct policy clauses, reason over structured data (formulary, provider directory, cost-sharing tables), and refuse to fabricate or guess when information is unavailable.

2.1 Key Pain Points Your System Must Address

- **Coverage Confusion:** Members are unsure whether a specific procedure or service is covered under their plan tier.
- **Prior Authorization Bottleneck:** Providers and members don't know upfront which procedures require pre-authorization and what criteria apply.
- **Claims Eligibility Uncertainty:** Agents re-read 80-page policy documents to verify whether a submitted claim meets eligibility requirements.

- **Deductible & Cost Opacity:** Members cannot easily calculate their expected out-of-pocket cost for a planned service.
- **Drug Coverage Lookup:** Members and pharmacists spend time determining whether a medication is on formulary and at what copay tier.
- **Plan Comparison at Enrollment:** Members comparing Bronze vs Silver vs Gold plans during open enrollment have no conversational way to get answers.

■ ■ Non-Negotiable Safety Constraint

The copilot must NEVER make a final coverage determination or give medical advice. Every response touching coverage eligibility must cite its source document and page, and include a disclaimer directing the member to contact their insurer for official verification. Failure to implement this will result in automatic deduction from the evaluation score.

3. Provided Dataset & Knowledge Base

The following synthetic (fictional) documents are provided to your team as the knowledge base. All data is created for training purposes only and does not represent any real insurer or real member. Your system must ingest, index, and query across all of these sources.

#	Document Name	Format	Size	Primary Use Case
1	Summary of Benefits & Coverage (SBC) Bronze / Silver / Gold Plans	PDF	12 pp × 3	Plan benefit lookup, cost-sharing Q&A;
2	Evidence of Coverage (EOC) Full Policy Document	PDF	85 pp	Coverage terms, exclusions, benefit limits
3	Prior Authorization Requirements List	PDF	18 pp	Which procedures need auth + clinical criteria
4	In-Network Provider Directory Excerpt	CSV	500 rows	Provider lookup, network status verification
5	Claim Submission & Eligibility Guidelines	PDF	10 pp	Claim eligibility rules, timely filing deadlines
6	Drug Formulary (Tier 1–5 Drug List)	CSV	1,200 rows	Drug coverage tier, copay amounts, restrictions
7	Preventive Care Schedule (ACA)	PDF	6 pp	Zero-cost preventive services reference
8	Member FAQ & Insurance Glossary	PDF	14 pp	Definitions: deductible, copay, coinsurance, OOP max

■ Ingestion Hint

Your system must handle both PDF and CSV formats. Consider how to attach meaningful metadata to each chunk (e.g., plan tier, document type, section name) to enable accurate filtered retrieval. Think about chunking strategy carefully — policy documents have nested sections and cross-references that naive chunking may break.

4. Expected System Architecture & Data Flow

The diagram below defines the five layers your system must implement and the numbered data flow steps your team must trace end-to-end. This is the target architecture — your implementation decisions (frameworks, models, deployment approach) are yours to make within these constraints.

Expected System Architecture & Data Flow

Participants must implement all five layers and the numbered data flow steps below

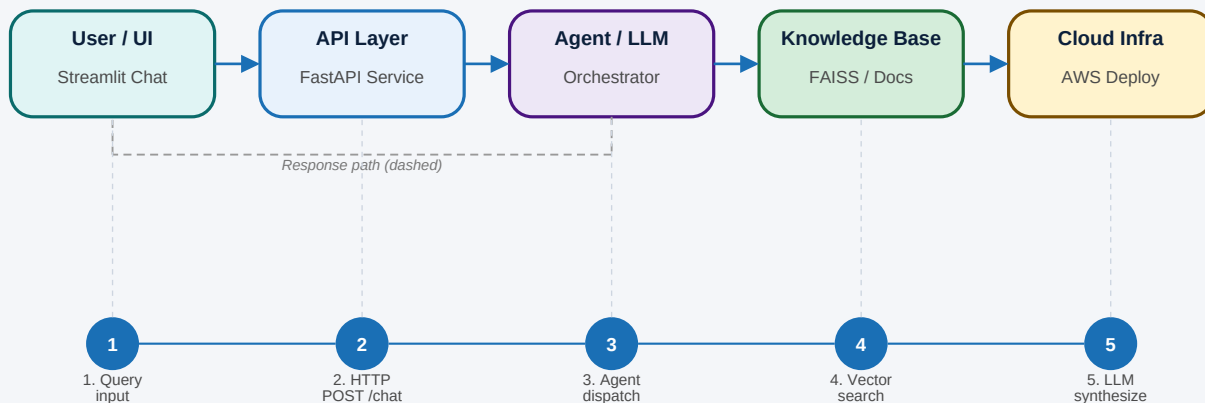


Figure 1: Five-layer reference architecture with end-to-end data flow steps

4.1 Layer Responsibilities

Layer	Component	Your Responsibility
User / UI	Chat Interface	A conversational front-end where members/agents type questions and receive answers. Must support session history, plan tier selection, and display of source citations.
API Layer	Backend Service	A REST API that accepts requests from the UI, validates inputs, routes to the agent, and returns structured JSON responses. Must expose a health endpoint and a chat endpoint.
Agent / LLM	Orchestrator	An LLM-based agent that interprets the user query, selects the appropriate retrieval or lookup tool, reasons over results, applies guardrails, and generates a grounded response.
Knowledge Base	Vector Store + Docs	All provided documents must be chunked, embedded, and stored in a vector index. The agent must retrieve relevant chunks via semantic search, with metadata filtering by plan tier.
Cloud Infra	Deployment Environment	The full stack must be containerized and deployed to AWS. A public URL must be accessible during the demo. Logs must be visible. Environment variables must be used for secrets.

4.2 Required Data Flow (5 Steps)

Your implementation must demonstrate all five steps of the data flow during the live demo:

- **Step 1 — Query Input:** User enters a natural-language question in the UI, optionally selecting their plan tier (Bronze / Silver / Gold).
- **Step 2 — HTTP POST /chat:** The UI sends the query, plan tier, and session ID to the API layer. The API validates the request before forwarding.
- **Step 3 — Agent Dispatch:** The agent receives the query, decides which tool or combination of tools to invoke (policy search, eligibility check, formulary lookup, cost calculator, or prior auth), and executes.

- **Step 4 — Vector Search:** For document-based queries, the agent retrieves the top-k most relevant chunks from the vector store, filtered by plan tier where applicable, and includes source metadata.
- **Step 5 — LLM Synthesize:** The LLM generates a grounded answer using retrieved context + conversation memory, passes it through guardrails, and returns a cited, safe response to the user.

5. Functional Requirements

Your system must fulfil the following capabilities. These are defined as business requirements — how you implement them is your team's design decision.

#	Requirement	Acceptance Condition
FR-01	Policy Coverage Q&A;	User asks any natural-language coverage question → system returns an accurate answer citing the source document, section, and page number.
FR-02	Plan-Tier Awareness	System must distinguish between Bronze, Silver, and Gold plan responses. Answers for one tier must not be applied to another.
FR-03	Claims Eligibility Check	Given a claim type and diagnosis code, the system must determine eligibility and return a structured reason with any applicable exclusions.
FR-04	Prior Authorization Lookup	Given a procedure code (CPT/HCPCS), the system must return whether pre-authorization is required and, if so, what the clinical criteria are.
FR-05	Drug Formulary Query	User asks about a drug by name → system returns coverage tier, copay amount, and any restrictions (quantity limits, step therapy).
FR-06	Member Cost Estimation	User describes a service and billed amount → system returns an itemized estimate of member liability (deductible, coinsurance, copay).
FR-07	Conversation Memory	Within a session, the system must remember prior context (e.g., 'for my Silver plan' stated earlier should apply to subsequent questions).
FR-08	Source Citation	Every response that references policy content must include: document name, section title, and page number.
FR-09	Guardrails	System must detect and block: PHI/PII in input, medical diagnosis requests, absolute coverage denials without citation, and prompt injection attempts.
FR-10	Escalation	When the system's confidence is low or the query is flagged as complex, it must inform the user and suggest contacting a human agent.

5.1 Non-Functional Requirements

- **Performance:** API must respond within 4 seconds at the 95th percentile under normal load.
- **Reliability:** FAISS index must persist across container restarts. Documents must not need re-indexing on every startup.
- **Security:** API keys, model credentials, and AWS credentials must be stored as environment variables — never hardcoded.
- **Observability:** All agent runs must be traceable. At minimum, you must show query, tool invoked, retrieved chunks, and final answer for each session turn.
- **Portability:** The full application must run via a single Docker command with no local environment dependency.

6. Evaluation Criteria & Scoring Rubric

Your capstone will be evaluated across five dimensions. The evaluation combines automated metric scoring (LangSmith + DeepEval) with a live demo assessment. Total: **100 points**.

Dimension	Weight	What Will Be Assessed	Minimum Threshold
RAG Accuracy	25 pts	Faithfulness (answer grounded in retrieved context, no hallucination) and Answer Relevancy scores on a 30-question test set.	Faithfulness ≥ 0.85 Answer Relevancy ≥ 0.80
System Completeness	25 pts	All 10 functional requirements (FR-01 to FR-10) demonstrated live. Each working requirement = 2.5 pts.	8 of 10 FRs must pass
Guardrail Effectiveness	20 pts	A set of adversarial test inputs will be injected: PHI in query, medical advice request, prompt injection, coverage hallucination trigger. Each blocked = 5 pts.	All 4 test cases blocked
Deployment & Ops	20 pts	Live accessible URL on AWS, Docker containerization confirmed, observability traces shown (LangSmith or equivalent), secrets handled via env vars.	Public URL must be live
Presentation & Architecture	10 pts	Team presents a clear architecture diagram and walks through the live system. Ability to explain design decisions under Q&A.;	Architecture must match implementation

6.1 Test Dataset Requirements

Your team must prepare a test dataset of **30 question-answer pairs** before running the evaluation pipeline. The dataset must cover all query categories:

Coverage Q&A; (policy documents)	10 questions	varied across all 3 plan tiers
Claims Eligibility	6 questions	mix of eligible, ineligible, and edge-case diagnoses
Prior Authorization	5 questions	procedures that require auth AND those that don't
Drug Formulary	5 questions	different tiers, generics vs brand names
Cost Estimation	4 questions	different services, different deductible states

■ Evaluation Tools

You must use LangSmith (or equivalent tracing tool) for run observability and tool-call logging. You must use DeepEval (or equivalent) to compute Faithfulness and Answer Relevancy on your 30-question test set. Both scorecard outputs must be shown during the demo.

7. Deliverables & Demo Checklist

At the end of Day 20, each group will present a live demo. The following items must be completed and demonstrable. Items marked **CRITICAL** are required for a passing grade.

CRITICAL	Chat UI loads and accepts natural-language queries with plan tier selector
CRITICAL	Policy Q&A; returns answer with source document name, section, and page number
CRITICAL	At least 4 of the 5 query capabilities (FR-01 to FR-06) demonstrated live
CRITICAL	Guardrail blocks at least one adversarial input during live demo (PHI or prompt injection)

CRITICAL	Application is live on a public AWS URL or EC2 IP during the demo window
REQUIRED	LangSmith (or tracing dashboard) shows traces with tool invocations for at least 5 queries
REQUIRED	DeepEval scorecard presented: Faithfulness and Answer Relevancy on 30-question test set
REQUIRED	Docker container confirmed: docker build and docker run commands demonstrated or evidenced
REQUIRED	Architecture diagram (minimum 1 slide) presented and matches the implemented system
BONUS	CI/CD pipeline or automated deployment script demonstrated (+5 bonus pts)
BONUS	Confidence scoring and graceful escalation to human agent demonstrated (+3 bonus pts)

8. Suggested Timeline (Day 19 & 20)

The following time-boxes are a guide. Your team decides how to allocate effort within each phase. All phases must be completed before the demo window on Day 20.

Phase	Focus Area	Time Box	Exit Condition
Phase 1	Data ingestion, document loading, vector indexing, metadata strategy	1.5 hrs	FAISS index built and queryable
Phase 2	Agent design, tool definitions, prompt engineering, memory setup	2.0 hrs	Agent answers at least 3 sample queries correctly
Phase 3	API development, endpoint design, validation, error handling	1.5 hrs	FastAPI running on localhost, /chat returns valid JSON
Phase 4	UI development, session state, plan selector, citation display	1.0 hrs	Streamlit UI connected to API and showing responses
Phase 5	Guardrails implementation, adversarial testing, escalation logic	1.0 hrs	All 4 required guardrail test cases blocked
Phase 6	Evaluation: test dataset, DeepEval run, LangSmith traces reviewed	1.0 hrs	Scorecard produced, Faithfulness ≥ 0.85
Phase 7	Docker build, AWS deploy, URL confirmed, demo rehearsal	1.0 hrs	Live URL accessible, demo rehearsed once

■ Final Reminder

This is a problem statement, not a solution guide. Your team must design the architecture, choose the implementation approach, and debug independently. Trainers will be available for unblocking critical issues only — not for step-by-step guidance.

Good luck, Group 01. Build something that works, is safe, and that you can defend.